

Towards an Open Format for Scalable System Telemetry

Teryl Taylor[†]

IBM Research

terylt@ibm.com

Frederico Araujo[†]

IBM Research

frederico.araujo@ibm.com

Xiaokui Shu

IBM Research

xiaokui.shu@ibm.com

Abstract—A data representation for system behavior telemetry for scalable big data security analytics is presented, affording telemetry consumers comprehensive visibility into workloads at reduced storage and processing overheads. The new abstraction, SysFlow, is a compact open data format that lifts the representation of system activities into a flow-centric, object-relational mapping that records how applications interact with their environment, relating processes to file accesses, network activities, and runtime information. The telemetry format supports single-event and volumetric flow representations of process control flows, file interactions, and network communications. Evaluation on enterprise-grade benchmarks shows that SysFlow facilitates deeper introspection into attack kill chains while yielding traces orders of magnitude smaller than current state-of-the-art system telemetry approaches—drastically reducing storage requirements and enabling feature-filled system analytics, process-level provenance tracking, and long-term data archival for cyber threat discovery and forensic analysis on historical data.

Index Terms—telemetry, open standard, data representation, system monitoring, threat detection

I. INTRODUCTION

Conventional perimeter monitoring offers limited coverage and visibility into workloads and attack kill chains, posing severe asymmetry challenges for defenders and impeding timely security analysis and response to resist known and emerging cyber threats. To cope with this visibility gap, comprehensive telemetry probes have recently been proposed to monitor system events and detect workload misbehaviors [1], [2], [3], [4]. However, current solutions expose system event information as low-level operating system abstractions (e.g., system calls), generating massive amounts of data and making it impractical and prohibitively expensive to store, process, and analyze such telemetry data, thus limiting analytics to rule-based approaches.

Existing system telemetry representations also suffer from unnecessary generalizations, trading interpretability and performance for the support of optional and custom attributes. This makes data schemas overly complex to ingest and process in realtime. Moreover, data formats lack support for concurrently tracking event provenance while being streamed live—critical for security investigations and root cause analysis, and most endpoint telemetry systems use proprietary data models, which impedes the implementation of standardized telemetry pipelines, forcing administrators to source monitoring data from multiple agents to satisfy regulatory requirements.

To overcome these disadvantages, we propose SysFlow as a new data representation for system behavior introspection for scalable analytics. SysFlow is a compact *open*¹ data format that

lifts the representation of system activities into a flow-centric, object-relational mapping that records how applications interact with their environment. It connects process behaviors to network and file access events, providing a richer context for analysis. This additional context facilitates deeper introspection into host and container workloads, and enables a stream of cloud workload protection use cases, including system runtime integrity protection, process-level provenance tracking, and long-term data archival for threat hunting and forensics.

SysFlow captures information about host, container, process, file, and network relationships and activities, supporting *single-event* and *volumetric flow* representations of process control flows, file interactions, and network communications. SysFlow reduces data collection rates by orders of magnitude relative to existing system telemetry sources, and lifts events into behaviors that enable forensic applications and more comprehensive analytic approaches. Furthermore, the open serialization format and libraries enable integration with open-source data science frameworks and custom analytics. This simplifies big data pipelines for systems akin to the way NetFlow [5] did for network analytics.

We also present a data processing pipeline built atop SysFlow. The pipeline provides a set of reusable components and APIs that enable easy deployment of telemetry probes for host and container workload monitoring, as well as the export of SysFlow records to S3-compliant object stores feeding into distributed security analytics jobs based on Apache Spark. Specifically, the analytics framework provides an extensible policy engine that ingests customizable security policies described in a declarative input language, providing facilities for defining higher-order logic expressions that are checked against SysFlow records. This allows practitioners to easily define security and compliance policies that can be deployed on a scalable, out-of-the-box analysis toolchain while supporting extensible programmatic APIs for the implementation of custom analytics algorithms—enabling efforts to be redirected towards developing and sharing analytics, rather than building support infrastructure for telemetry.

SysFlow's collection probe has been optimized to incur minimal performance overheads and does not require program instrumentation or system call interposition for data collection, therefore having a negligible impact on monitored workloads. The implementation has been validated under multiple stress test profiles. We describe its components in detail and demonstrate that the ability to continuously collect and store comprehensive system event information is critical for the identification of advanced and persistent threats, security vulnerabilities, and performing threat hunting tasks.

Our contributions can be summarized as follows:

- A compact data format and new flow analysis that records the interactions of processes and containers with their environment,

[†]Both authors contributed equally to this research.

¹<https://github.com/sysflow-telemetry>.

providing an object-relational structure that enables many security and systems monitoring applications.

- A data processing language and pipeline for consuming and analyzing SysFlow records at scale.
- A capability analysis of SysFlow on its ability to express attack tactics, techniques, and procedures described in the MITRE ATT&CK framework [6].
- A comprehensive evaluation of state-of-the-art system telemetry solutions based on popular, enterprise-grade container applications and standard industry benchmarks.

II. OVERVIEW

A. Computer Telemetry

Computer telemetry can be broadly categorized into network and system telemetry. Network telemetry involves installing a passive tap at the perimeter of the network or on an endpoint to collect packet data. System telemetry includes application logging, program tracing, system call collection, and runtime metric gathering at endpoints.

Network Telemetry. The excessive cost of full packet capturing has led researchers to develop techniques to reduce its storage burden. Maier et. al [7] found that several days of traffic could be stored by collecting raw PCAP (Packet Capture) data up to a configurable byte cutoff per connection while retaining a complete record for the majority of network connections. This *time machine* was combined with the Bro (Zeek) intrusion detection system [8] to enable real-time streaming analytics for security and retroactive data enrichment.

While storing packet headers has also been used to reduce storage requirements, the most popular network telemetry system is NetFlow [5], which aggregates packets from network sessions into single records that contain the 5-tuple identifier of the sessions combined with volumetric information. NetFlow provides orders of magnitude compression over packet collection and has been a staple for network-based analytics for the past two decades. Extensions have been proposed to improve the format [9], [10], augmenting it with more attributes [11], and building more scalable probing mechanisms [12].

However, despite its popularity, network telemetry only provides *partial* visibility into system workloads, requiring a host-level monitoring source for capturing system behaviors.

System Telemetry. While research on network telemetry has focused on data representations for scalable storage and processing, system telemetry is still dominated by approaches that operate on raw system call data [1], [2], [3], [4], [13], [14], [15], [16], [17]. This causes many issues in terms of data collection size, efficiency, and performance. To alleviate these challenges, KCAL [3] improves data collection speeds through a kernel cache and aggressive system call filtering, while LogGC [16] leverages a post-processing algorithm for pruning unreachable objects. SPADE [13] defines a graph model for data provenance collection and storage, and ProTracer [4] combines both logging and unit-level tainting to achieve cost-effective provenance tracing. Fmeter [18] reduces data pressure by creating feature vectors of system call counters to perform indexable searches. System call analysis is also used in debugging for deterministic process replay [19], [14], which

typically requires the injection of special libraries to record time and event features [20].

Among the most popular system call monitors is Linux Audit [1], which has been the de facto standard for GNU/Linux tracing since Kernel version 2.6, being used by many Linux distributions, and tools such as osquery [21]. Our evaluation (§V) shows that Audit is not ideal for comprehensively collecting system call data at cloud scale—its kernel probe loses events under load, telemetry output is too large, and native container support is not yet upstream. Sysdig [2] scales better than Audit for collecting system call information; however, it yields significant data footprints, making it intractable to store the telemetry stream, and therefore reducing data processing capabilities to low-overhead rule-based techniques. The telemetry exported by these monitors also make it extremely difficult to efficiently build event provenance graphs or perform cyber threat hunting. To overcome these issues, commercial endpoint monitoring solutions adopt their own proprietary formats. However, this requires the installation of multiple monitors to support various security products, wasting important cloud computing resources, and making integration across security products difficult.

Limitations in Data Representation. While many existing endpoint telemetry solutions provide low-level system information that can be harnessed for attack reasoning, provenance tracking, and performance analysis, they often suffer from inefficiencies in tracing [22], [23] and excessive data redundancy [3], exacerbating the challenges of collecting, storing, and processing such telemetry data. Several techniques have emerged in an attempt to mitigate data requirements, including folding sub-groups of system events according to their semantic meaning [24], [25] and pre-processing events at the probe level to reduce data redundancy [3]. However, none offer a comprehensive telemetry specification that satisfies the performance and data representation requirements of host-level telemetry.

Most recently, a new ontology for system telemetry has been developed in the context of the DARPA Transparent Computing (TC) program [26], which investigated how security analysts would benefit given access to a *complete* and *high-fidelity* recording of computations across every endpoint in a compromised computer network. The program brought together teams from academia and industry to work on the generation and analysis of real-time monitoring of computations on an array of system environments and architectures. TC defined a Common Data Model (CDM) for communicating system behaviors and for representing data provenance and information flow semantics based on system calls.

However, despite its many innovations on system monitoring [14], [3], [27], cyber reasoning methodologies [28], and automated analytics [29], [30], TC engagements were conducted in small networks. As originally envisioned, TC does not scale to large enterprise environments due to the sheer volume of fine-grained telemetry data it generates, and the difficulties to transport, ingest, process, and store data in CDM, which is not designed for compactness. In addition, CDM creates excessive processing complexity through the support of hundreds of optional attributes, allowing it to be interpreted differently by various users. CDM advances system telemetry but has limited support for efficient stream processing for real-time security and compliance analysis in large-scale deployments.

B. System Telemetry Requirements

The lessons we learned through the development of a cyber reasoning platform [28] for DARPA TC informed many practical requirements for effective system telemetry. In particular, our experiences with system event analysis and the CDM format revealed that (1) system telemetry is often collected at system call granularity—making it impractical to scalably process and store historical telemetry data, (2) employed schemas offer limited support for data provenance; collection stacks do not support provenance tracking and data streaming simultaneously, and (3) existing data representations trade interpretability for flexibility through optional and custom attributes, hindering processing performance. In addition, telemetry systems often use proprietary data models, forcing administrators to source telemetry data from multiple agents, wasting resources, and making integration more challenging; this issue is exacerbated by the lack of open interfaces for consuming the collected data.

To cope with these shortcomings, an efficient system telemetry approach must offer a combination of lightweight, non-intrusive event collection with a data representation that is open, non-redundant, compact, and capable of expressing essential system properties and behaviors. Such representation must also abstract away noisy low-level system events while preserving the semantics of system executions, to enable a telemetry pipeline that is conducive of dense, long-term archival of system traces, and efficient reasoning over historical system behaviors. Furthermore, the format must inherently support linkages of system behaviors with processes, users and containers to enable data provenance and process control flow graphs on streaming data.

C. Towards System Flows

To achieve these goals, we envision a system telemetry stack that is capable of processing and summarizing system events into process behaviors while preserving system execution semantics, analogous to how NetFlow aggregates raw packet information. Towards this end, we introduce SysFlow as a new flow-based data specification for system monitoring. NetFlow played a pivotal role in scaling network-based analytics by condensing packet information into a much smaller flow data; furthermore, NetFlow was widely adopted by the community because it is an open source format, and simple to use. SysFlow adapts and extends such flow abstraction to model systems entities and interactions, collecting metadata from system calls and grouping sequence of events sharing the same properties.

Figure 1 shows how SysFlow can be used to uncover a targeted attack in which a cyber criminal exfiltrates data from a cloud-hosted service. During reconnaissance (step 1), the attacker detects a vulnerable node.js server that is susceptible to a remote code execution attack exploiting a vulnerability in a node.js module (e.g., CVE-2017-5941 [31]). The attacker exploits the system using a malicious payload (step 2), which hijacks the node.js server and downloads a python script from a remote malware server (step 3). The script contacts its command-and-control server (step 4), and then starts scanning the system for sensitive keys, eventually gaining access to a sensitive customer database (step 5). The attack completes when data is exfiltrated off site (step 6).

While conventional monitoring tools would only capture streams of disconnected events, SysFlow can connect the entities and effects of each attack step on the system. For example, the

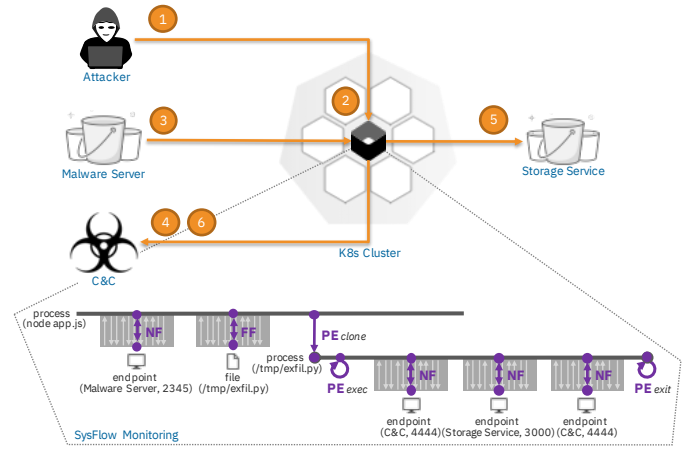


Fig. 1: Using SysFlow to uncover a targeted attack.

highlighted SysFlow trace records and reveals precisely the steps in the attack kill chain: the node.js process is hijacked, and then converses with a remote malware server on port 2345 to download and execute a malicious script (exfil.py), which is eventually executed and starts an interaction with a command-and-control server on port 4444 to exfiltrate sensitive information from the customer database on port 3000.

This example showcases the advantages of applying flow analysis to system telemetry. SysFlow provides visibility within host environments by exposing relationships between containers, processes, files, and network endpoints as events (single operations) and flows (volumetric operations). For example, when the node.js process clones and execs into the new process, these tasks are recorded as process events (PE), and when a process communicates with a network endpoint or writes a file, these interactions are captured and summarized using compact file (FF) and network (NF) flows. The result is a graph-like data structure that enables precise reasoning and fast retrieval of security-relevant information, lifting system calls into a higher-level, compact representation that enables defense automation, scalable analytics, and forensics.

III. SYSFLOW

SysFlow is an open system telemetry format that lifts system call events into behaviors that describe how processes interact with system resources. Its design emphasizes security visibility, with increased importance on linking process behaviors for threat hunting, reducing data footprints for forensic storage, and providing a rich list of attributes to identify malicious techniques, tactics, and procedures (TTPs). To support these goals, SysFlow aggregates related system calls into compact summarizations that are time-bound and contain a vast number of carefully selected attributes and statistics, ideal for a wide variety of analytics and forensics. The format's object-relational abstraction inherently encodes the graph-like structure of a process interacting with its environment, which is ideal for gaining contextual information about attacks, and supporting threat hunting. Figure 2 shows an overview of SysFlow's main objects and relations, which are categorized as *entities*, *events*, and *flows*.

A. Entities

Entities represent components or resources that are monitored on a system. The current specification supports four types of

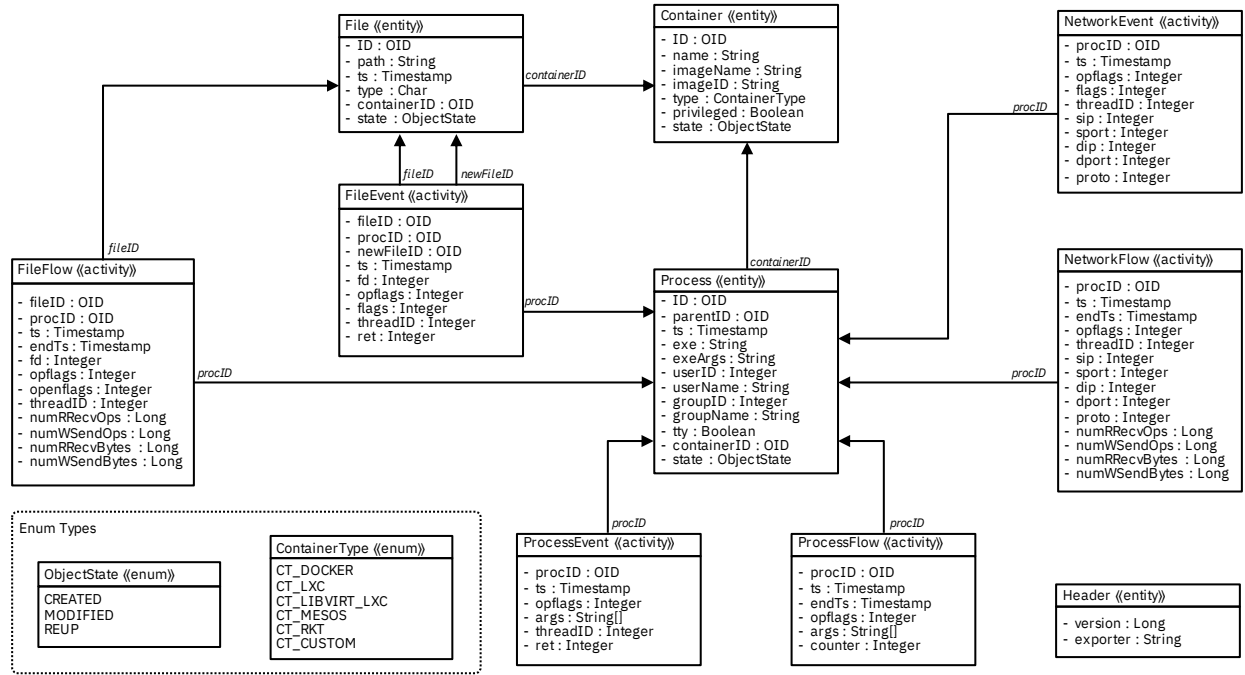


Fig. 2: Overview of SysFlow's object-relational view.

entities: *header*, *container*, *process*, and *file*. The header object contains runtime and meta-data information about the host system being monitored, including hostname, distribution, kernel version, and SysFlow's specification version.

Entities (except the header) contain a timestamp, an object ID and a state along with other entity-specific attributes (e.g., container name, exe, pid). The timestamp attribute is used to indicate the time at which the entity was exported to the captured trace. Each entity has an object ID to allow events, flows and other entities to reference each other without having duplicate information stored in each record and to support rapid process graph building on SysFlow data streams. Object IDs are not required to be globally unique across space and time. In fact, the only requirement for uniqueness is that no two objects managed by a SysFlow exporter can have the same ID simultaneously. Entities are always written to the trace file before any events, and flows associated with them are exported. Since entities are exported first, each event, and flow is matched with the entity (with the same ID) that is closest to it in the file.

The container object represents a container-based workload, such as Docker or LXC, and is important to support the telemetry collection of modern cloud environments based on technologies, such as Kubernetes [32], Docker Swarm [33], and OpenShift [34]. It contains information about its image, name, type and ID. Central to the format is the process object, which contains a reference ID to the container object, as well as its parent process object ID. This greatly facilitates the tasks of relating processes to their containers and reconstructing the entire process tree.

B. Events & Flows

Entity behaviors are modeled as *events* or *flows*. These behaviors typically involve a process interacting with some resource (e.g., file, network, or another process) and are created from system call tracing. Events represent important *individual behaviors*

of an entity that are broken out on their own due to their importance, their rarity, or because maintaining operation order is important. An example of an event would be a process *clone* or *exec*, or the deletion or renaming of a file. By contrast, a flow represents a *volumetric aggregation* of multiple events that naturally fit together to describe a particular behavior. For example, we can model the network interactions of a process and a remote host as a bidirectional flow that is composed of several events, including *connect*, *read*, *write*, and *close*. During extensive experimentation with raw system call data collection, we observed that file reads/writes, network sends/receives, and process threads creations are the main contributors to large trace sizes in conventional system telemetry approaches. This motivated our key insight that *flows* can significantly reduce data footprints without loss of visibility. Flows are summarized by thread ID, meaning that a flow started by one process or thread and passed to another, will generate two flow records.

All events and flows contain a process object ID, a timestamp, a thread ID, and a set of operation flags (op flags). The process object ID represents the process entity on which the event or flow occurred, while the thread ID indicates which process thread was associated with the event. Behaviors are encoded in the op flags and represented in a single bitmap which enables multiple actions to be easily combined. The difference between events and flows is that an event will have a single bit active, while a flow could have several. Flows represent a number of events occurring over a period of time, and thus have a set of operations (multiple bits set in the op flags), a start and an end time. One can determine the operations in the flow by decoding the op flags. A flow also contains a set of counters to record the number of operations over the duration of the flow.

A flow is started by any supported flow operation and is exported on (1) an *exit* event of its owning process, (2) a *close* event signifying the end of a connection or file interaction,

TABLE I: Operations breakdown by event and flow type.

Event/Flow	Supported Operations
Process Event	clone (process), exec, exit (process), setuid, setgid
Network Event	bind, listen
File Event	mkdir, rmdir, unlink, symlink, link, rename, chmod, chown, (u)mount
Process Flow	clone (thread), exit (thread)
Network Flow	accept, connect, send, recv, shutdown, close
File Flow	open, read, write, setns, mmap, close

or (3) a preconfigured timeout. After a long-running flow is exported, its counters and flags are reset; if no flow activity is observed during the timeout period, that flow is not exported.

Operations. Table I shows the list of operations supported by SysFlow. It comprises 31 operations that map to one or more system calls. Note that SysFlow does not seek to provide coverage of every system call supported by the kernel—it would be far too costly to store and analyze. Rather, the goal is to provide a low-noise, *semantics-preserving* telemetry source that records how processes affect their environment. These types of behaviors are extremely important for security and performance analyses including identifying cyber attack tactics, techniques and procedures, and are highly compressible.

The underlying operational abstraction reveals that event and flow objects can represent process (control), file (access) and network (interaction) activity. We detail these next.

Process Activity. Process events and flows represent behaviors that modify a process’ control flow. The difference between a process event and a flow, is that the process event is emitted every time there is a modification to the process (e.g., change user ID), a process `exit` is observed, or a new process is spawned. By contrast, a process flow keeps a summary of new threads created and destroyed over a time period. Process flows are created on `clone` events, and only emitted when the flow expires (after a timeout), or when the process exits.

Table II shows a sample of the SysFlow trace discussed in §II-C, which describes the launching of a node.js server, acceptance of an incoming HTTPS request, dropping and execution of a malicious script followed by the termination of both applications running in container `node-js`. Rows highlighted in gray correspond to suspicious behavior. Records #1 and #8 captures the process events corresponding to the execution of process `node` and suspicious script `exfil.py`, respectively. Moreover, record #9 shows the execution of package installer `apt`, whose parent PID (PPID) corresponds to the suspicious script’s PID. Finally, records #10, #13 and #16 signal that the processes have terminated. Note that the trace only displays a subset of the record attributes due to space constraints.

File Activity. File events and flows represent process interactions with files, pipes, UNIX sockets, and devices on a system. File events focus on self-contained system calls that create, delete, or rename files and directories. Conversely, file flows summarize all the operations executed on the same file handle. A file flow will typically begin on a file `open`, summarize operations on that handle, and then expire on a file `close`, or after a timeout, after which flow counters are reset. File flows keep track of the number of `read` and `write` operations on a file, as well as the number of bytes read and written.

To illustrate, record #2 denotes a file flow in which the `node` process reads 832 bytes from the `libc.so` shared library in one `read` operation (1:832). The `O R C` operation flags indicate that `node` `opened`, `read`, and `closed` the shared object. The process then creates a `/tmp/log` directory as captured by the file event in record #3. The process continues by writing to `app.log` over a 2-minute period, recorded as three file flows (#4, #14–15). The first flow shows when the file is opened and 8000 bytes are written using 100 `write` operations. The flow expires, and a *continuation* flow is created, which records the next 8000 bytes written. Finally, the second flow expires, and a third flow is exported with the remaining writes and the final `close` operation. In between writes to the log file, the node server is hijacked, and an `exfil.py` is written as a file flow (#7) and then executed.

Network Activity. Network events and flows are analogous to their file counterparts but work on sockets rather than files. A network flow records all interactions of a process with a local or remote IP address containing the same 5-tuple (IP addresses, ports, proto) as Cisco NetFlow. Network flows also contain counters for the number of bytes sent and received similar to bidirectional NetFlows.

Network flows differ from NetFlows in that they operate at the transport layer, by monitoring and summarizing network-related system calls such as `accept`, `connect`, `send`, or `recv`. It requires the application to explicitly interact with a remote/local endpoint in order to be generated. By contrast, NetFlow operates at the Internet/Network (or packet) layer. This means that NetFlow has the concept of packets and TCP Flags (which network flows do not), and can record traffic from remote hosts that is ignored by the local system.

While network flows do not record certain remote traffic patterns like scanning activity, they record active process connections and allow network traffic to be tied to process information as well as file information, providing a more granular and comprehensive data source for security and performance analytics. Record #5 is a network flow representing the attacker’s connection to the `node` server. The server `accepts` the connection from the attacker, `sends` and `receives` data, and finally `closes` the connection, as denoted by the operation flags. After `exfil.py` is written and executed, it connects to a storage service on port 3000 (#11) and begins data exfiltration to a command-and-control server (#12) on port 4444.

C. Extended Attributes

Certain attributes, such as domain names and file hashes, are not obtainable through system calls but can be gathered through other mechanisms to enrich SysFlow records. These are captured as extended attributes and are not depicted in Figure 2.

IV. DESIGN & IMPLEMENTATION

Our implementation comprises a collection stack and a data processing pipeline for SysFlow built atop the Linux operating system. We also describe a cloud-native telemetry architecture that can store and analyze SysFlow data at scale. Other telemetry stacks and operating systems such as Windows can also benefit from utilizing SysFlow’s open schema, but a more detailed discussion is beyond the scope of this paper.

TABLE II: Simplified SysFlow records (PE=Process Event, FF=File Flow, FE=File Event, NF=Network Flow).

#	Type	Process	PPID	PID	Op Flags	Start Time	End Time	Resource	Reads	Writes	Cont ID
1	PE	node app.js	1887	21847	EXEC	4/10T16:47			:	:	node-js
2	FF	node app.js	1887	21847	O R C	4/10T16:47	4/10T16:47	/lib/gnu/libc.so	1:832	:	node-js
3	FE	node app.js	1887	21847	MKDIR	4/10T16:47		/tmp/log	:	:	node-js
4	FF	node app.js	1887	21847	O W	4/10T16:47	4/10T16:48	/tmp/log/app.log	:	100:8000	node-js
5	NF	node app.js	1887	21847	A SR C	4/10T16:48	4/10T16:49	<IP _{attacker} >:3522 – 172.30.10.2:443	1:80	2:980	node-js
6	NF	node app.js	1887	21847	C SR C	4/10T16:48	4/10T16:49	172.30.10.2:8353 – <IP _{Malware} >:2345	3:4355	1:94	node-js
7	FF	node app.js	1887	21847	O W C	4/10T16:48	4/10T16:48	/tmp/exfil.py	:	6:4250	node-js
8	PE	/tmp/exfil.py	21847	21849	EXEC	4/10T16:48			:	:	node-js
9	PE	apt install pip	21849	21851	EXEC	4/10T16:48			:	:	node-js
10	PE	apt install pip	21849	21851	EXIT	4/10T16:48			:	:	node-js
11	NF	/tmp/exfil.py	21847	21849	C SR C	4/10T16:48	4/10T16:48	172.30.10.2:8356 – <IP _{Storage} >:3000	2:165	1:34	node-js
12	NF	/tmp/exfil.py	21847	21849	C SR C	4/10T16:48	4/10T16:48	172.30.10.2:8357 – <IP _{C&C} >:4444	1:46	2:188	node-js
13	PE	/tmp/exfil.py	21847	21849	EXIT	4/10T16:48			:	:	node-js
14	FF	node app.js	1887	21847	W	4/10T16:48	4/10T16:49	/tmp/log/app.log	:	100:8000	node-js
15	FF	node app.js	1887	21847	W C	4/10T16:49	4/10T16:49	/tmp/log/app.log	:	50:4000	node-js
16	PE	node app.js	1887	21847	EXIT	4/10T16:50			:	:	node-js

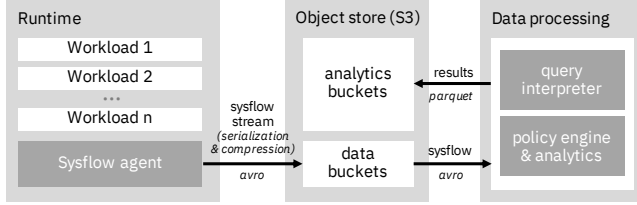


Fig. 3: SysFlow data processing pipeline.

A. Data Collection

We built our collector using Sysdig’s core system telemetry libraries (v0.26.7) [2]. Sysdig uses a kernel module and tracepoints to dump system call information into a ring buffer, which is in turn processed by Sysdig’s core libraries². We chose Sysdig because it is an open source project with strong community support, has a powerful API that supports container monitoring natively, and provides all the attributes SysFlow requires. Other telemetry options such as Linux Audit do not support container as first class objects. Our implementation is ~4k lines of C++ code. The telemetry stream is serialized using Apache Avro [35] (see §V-B for an empirical discussion of serialization formats).

B. Cloud-native Telemetry Pipeline

To enable efficient processing and analysis of SysFlow data, we implemented a cloud-native telemetry pipeline for large-scale system analytics. The pipeline, shown in Figure 3, deploys and replicates the SysFlow agent (collector and exporter) to every Kubernetes [36] node (daemonsets) of monitored cloud clusters, using Helm charts [37] for deployment automation. The agent streams SysFlow objects to files on each node, which are subsequently pushed, at predefined intervals, to buckets in a distributed S3-compliant cloud object store. Once exported, both microservice- and Spark-based analytics are employed to continuously process telemetry data.

Querying & Analytics. Data introspection is made possible via a set of programmatic and declarative APIs offered to telemetry consumers via a simple, yet powerful processing language that supports both alerting and enrichment of telemetry records. We used these APIs to implement language runtimes and a declarative policy engine for both Spark (Java) and Python, and are exploring

scalable machine learning approaches to detect security incidents in cloud environments. Note that since objects are serialized Avro, we can natively support analytics in any major programming language, as well as big data platforms.

Processing Language. We designed a domain-specific language along with its distributed runtime for processing SysFlow traces at scale. The language is declarative and embeds a simple predicate logic extended with operators, lists and macro definitions. For explanatory precision, a simplified version of the language is shown in Figure 4.

Policies P are sets of lists, macros, and rules. A *list* is an array of values (e.g., names of common package installers). A *macro* is a named condition, denoted c . A *rule* defines an action, which can be to `match` a stream of SysFlow records with c (and optionally `show` the records $\bar{sf}$ satisfying the constraints defined in c), or to `tag` the list of matched records with label set $\bar{u}$. Conditions are defined as logical expressions predicated over primitive SysFlow attributes (e.g., $sf.proc.exe$, $sf.file.path$) and attribute extensions defined in the language. For example, one useful attribute extension is the ability to predicate over ancestry chains, like in the rule:

```
match sf.proc.achain(2) in shell_binaries
```

which matches all records with a shell as grandparent.

C. Automated Attack Detection

In this section, we return to the remote attack presented in §II-C. and investigate how the policy language and analytics pipeline can be used to detect such an attack and perform threat hunting. SysFlow collectors monitor the container workloads in the targeted environment and push SysFlow records into the analytics pipeline described in §IV-B. The pipeline is equipped with a distributed policy engine based on SysFlow’s processing language. Any container actions that violate a security policy trigger an alert.

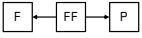
Security Policies. A typical policy is that containers requiring an update must be rebuilt and redeployed. Such policy must therefore prohibit someone from executing a system package installer (e.g., `yum`, `apt`) inside a running container. This policy can be defined as shown in Rule 1, which alerts on the execution of any software management tool.

Rule 1: Detection of package manager execution.

```
match sf.proc.exe pmatch (apt, yum, dnf, ...)
```

²Sysdig also supports eBPF probing since version 0.26.4.

TABLE III: MITRE ATT&CK TTPs that can be observed in SysFlow streams and expressed as behavioral policies; P=Process, F=File, PE=Process Event, PF=Process Flow, FE=File Event, FF=File Flow, NE=Network Event, NF=Network Flow.

Type	Behavior	Expressible TTPs
PE		process injection, process modification, process discovery, hijack execution flow, event trigger execution, command and scripting interpreter abuse, native API exploit, scheduled task/job, software deployment tools, account manipulation, modprobe invocation, application instability/crash detection, spikes in application activity, sudo execution, kill signals to security applications, compiler executions, package installation, history tampering (HISTCONTROL), library injection (LD_PRELOAD), regexes in grep commands, local/logged user listing, filesystem scanning, account discovery, group execution, network discovery, remote file copy execution, compression binaries execution, screen capturing execution, system shutdown/reboots
PF		spikes in application activity, thread instability/trashing behavior
FE		filesystem access control modification, mknod execution, file deletions, file metadata modification, hidden directory creation, spikes in application activity, /var/log deletion
FF		account discovery, file download, protected file writes, configuration file modification, scheduled files/crontab modification, file name injection, bash profile modification, /etc/passwd modification, hidden file creation, read from .ko files, installation of new classes/libraries, new classes/libraries loaded in application startup, system init process modification, sudoers file modification, executable file modification, bash history modification, root certificate modification, credentials scanning, unusual access to cached password data
NE		unusual port bind and listen operations
NF		network sniffing, communication with new network device, interactions with known malicious site, strange SSH login attempts, failed SSH logins, beaconing/periodic patterns in network traffic, suspicious correlation between process and network activity, unusual access patterns of shared drives or remote repositories, unusual ports, spike in network activity

<i>policies</i>	$P ::= (list \mid macro \mid rule)^+$
<i>lists</i>	$list ::= v ::= \bar{u}$
<i>macros</i>	$macro ::= v ::= c$
<i>rules</i>	$rule ::= match\ c \ [\ show\ \bar{sf} \]$ $\mid tag\ c\ with\ \bar{u}$
<i>conditions</i>	$c ::= e \ (or\ e)^*$ $e ::= t \ (and\ t)^*$ $t ::= v \mid not\ t \mid a \Diamond_u \mid a_1 \Diamond_b a_2$
<i>unary ops</i>	$\Diamond_u ::= exists$
<i>binary ops</i>	$\Diamond_b ::= in \mid pmatch$ $\mid contains \mid startswith \mid \dots$ $\mid typical\ comparison\ operators$
<i>atoms</i>	$a ::= u \mid v \mid sf$
<i>variables</i>	v
<i>values</i>	$u ::= values\ of\ the\ underlying\ language$
<i>records</i>	$sf ::= sysflow\ record\ attributes$

Fig. 4: SysFlow simplified processing language syntax.

Rule 1 alerts the bootstrap phase of the attack, where apt (see Table II) is invoked for the preparation of the Python runtime. The policy engine implements rules that inspect container integrity, access control operations, known attack TTPs, and application-specific misbehavior during container lifecycle.

Policy Expressiveness. SysFlow can express security-relevant behaviors succinctly and without the need to record raw system call information. Table III summarizes our analysis of the MITRE’s ATT&CK [6], in which we show how different SysFlow components can be used to characterize the vast majority of tactics, techniques, and procedures (TTPs) described in the framework (e.g., Rule 2). SysFlow’s graph-relational structure captures system events in the context of other surrounding events, which enables the characterization of behavioral patterns and provenance chains. Our flow-based abstractions occlude inter-arrival times and individual system call granularity for certain operations; however, we find this to be an acceptable trade-off for most analyses. Moreover, in circumstances where more granularity is required, analysis can be performed on streamed system call data, with the

raw telemetry stored in SysFlow format for evidence preservation and forensics.

Rule 2: Tagging of MITRE’s T1087 (*account discovery*).

$tag\ sf.file.path\ in\ (/etc/passwd, /etc/shadow, \dots) \text{ with } [T1087]$

Cyber Threat Hunting. In security operation centers, analysts dequeue alerts from automated detectors, triage them, search for their context, and reason about them for the bigger picture of multi-phase attack campaigns. The system telemetry provided by SysFlow enables comprehensive context completion, provenance tracking, and reasoning to hunt threats. For each alert, we implemented rules to automatically extract context from related entities, such as *call chains* (Rule 3) and *loaded files*.

Rule 3: Show ancestry chain of a process.

$match\ sf.proc.exe\ contains\ exfil.py\ show\ sf.proc.achain$

More advanced capabilities can be coded by threat hunters to explore the neighborhood of alerts, mine connections, and track data provenance. In our pipeline, analysts can use a SQL interface as the query language, and the queries are translated into the SysFlow processing language. Starting from the prohibited execution of apt and the beaconing alerts, the analyst quickly locates the node.js worker, which is on both the call chain of the Python script that exfiltrates data and the call chain of the apt process that precedes the script execution. The attack is fully revealed when activities of processes on the call chain are displayed, which includes environment setup with pip, object store search, and data retrieval from the storage service.

V. EVALUATION

To demonstrate the efficacy of SysFlow, we monitored popular cloud services under strenuous benchmark conditions and measured their throughput, resource utilization, and performance impact on the workloads. We also compared our collector implementation with two mainstream telemetry frameworks: go-audit [38] and Sysdig [2]. In summary, our collector generates *orders of magnitude less data* than the syscall-granular alternatives, especially when

TABLE IV: Benchmarks used in the evaluation.

Workload	Benchmark	Settings
httpd_prefork	ab [41]	<i>c</i> : 1,5,10,25,50 <i>t</i> : 60s <i>n</i> : 100M
httpd_worker	ab [41]	<i>c</i> : 1,5,10,25,50 <i>t</i> : 60s <i>n</i> : 100M
httpd_event	ab [41]	<i>c</i> : 1,5,10,25,50 <i>t</i> : 60s <i>n</i> : 100M
minio	Wasabi S3 BM [42]	<i>threads</i> : 1, 5, 10 <i>dur</i> : 15m <i>file size</i> : 5, 64, 256 (MB)
mysql	TPC-H HDB [43]	default
postgres	TPC-C HDB [43]	default
redis	TPC-C HDB [43]	default
hadoop	HiBench [44]	<i>scale</i> : large, <i>tests</i> : wordcount, sql scan, pagerank
matrix_mult	m-thread bench [45]	<i>benchmark</i> : 3, <i>matrix size</i> : 5000x5000, 1 thread per row

monitoring I/O-heavy applications, such as databases and object stores. Furthermore, our collector does not affect the performance of monitored workloads, while using less than one vCPU under the heaviest workload profiles.

A. Experimental Setup

Experiments were conducted on 4 virtual machines (VMs) hosted on a popular cloud platform, each containing 48 2.0GHz cores, 192 GB of RAM, 2 TB of disk space, and running Ubuntu 18.04. Two VMs were used as load testers while the other two hosted the targeted workloads; workloads (except Hadoop) were deployed as docker containers.

To cope with the magnitude of the experiments, we built an automated evaluation framework and testing harness based on Ansible [39] playbooks. Ansible allowed us to deploy both workload and monitoring containers on the desired machines while kicking off load tests and gathering performance statistics. psrecord [40] was used to measure CPU and memory usage on all workloads and collectors.

Sysdig was configured to use zlib block compression on its data, while SysFlow used the deflate codec for block compression. Furthermore, SysFlow was configured to timeout and export long-running file and network flows every 30 seconds. All three telemetry sources filter for the same set of system calls. Finally, go-audit does not support filtering by container natively. As a result, we ran containers and processes under a special user ID, and filtered system calls on that user.

B. Performance Benchmarks

Table IV shows the list of workloads and benchmarks used in the evaluation. We chose these applications for their wide popularity in cloud-based deployments and plurality in terms of roles and software architecture (e.g., process intensive, thread intensive, file intensive, network intensive), enabling the creation of very diverse telemetry footprints.

We also selected a set of standard industry benchmarks that tax workloads with higher than normal traffic patterns to demonstrate the capabilities of the collectors under intense pressure. Benchmarks include data processing (TPC-C), decision support (TPC-H), web traffic generation (Apache Benchmark), S3 object creation, retrieval and deletion, multi-threaded matrix multiplication, as well as big data analytics.

Collector CPU and Memory Utilization. We measured the CPU and memory utilization statistics for all the collectors during the benchmarks. Figure 5³ shows that the heavier the workload, the

³Note that a subset of our tests are shown due to space constraints.

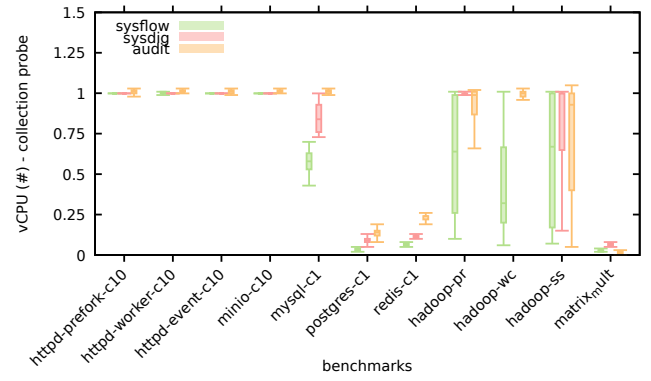


Fig. 5: CPU utilization. The benchmark name suffixes denote the number of concurrent clients *c* for that benchmark.

TABLE V: Trace sizes (MB) and parenthesized object counts ($\times 10^3$) across benchmarks.

Benchmark	SysFlow	Sysdig	Go-Audit
httpd_prefork_c10	11 (818.59)	62 (7,451.43)	78 (194.11)
httpd_worker_c10	7.9 (629.31)	58 (6,399.76)	80 (193.28)
httpd_event_c10	7.8 (642.35)	53 (5,990.95)	75 (184.80)
mysql_bs	0.1 (1.42)	3735 (139,104.20)	794 (1,888.74)
mysql_c1	0.09 (0.19)	2592 (156,002.41)	327 (786.26)
postgres_bs	0.07 (1.53)	7.8 (825.19)	188 (400.69)
postgres_c1	0.22 (7.08)	169.2 (9,314.71)	2000 (4,291.52)
redis_bs	0.1 (6.04)	160.8 (8,687.36)	2500 (4,253.15)
redis_c1	0.15 (9.10)	68 (15,240.27)	4100 (8,309.04)
minio_c10	19.5 (752.49)	552 (26,241.50)	966 (2,188.92)
hadoop_pr	16.4 (626.51)	234 (10,945.46)	1700 (2,469.77)
hadoop_wc	7.1 (259.92)	45 (2,609.22)	875 (1,239.34)
hadoop_ss	6.2 (225.86)	42 (2,044.28)	807 (1,128.15)
matrix_mult	0.008 (0.015)	1.2 (30.1)	5.2 (11.49)

more CPU the collector uses, and all collectors use around the same amount of CPU for CPU-intensive benchmarks, such as the Apache Benchmark. Also, given that SysFlow is built on top of Sysdig's system call probe, the two collectors perform similarly in terms of CPU and memory usage, indicating that flow aggregation does not increase resource usage significantly. In fact, SysFlow outperforms the other collectors on the database benchmarks because its output is reduced by several orders of magnitude more than the other sources. As a result, our collector is not doing block compression or intensive I/O like the other collectors. In general, CPU usage varies wildly for Hadoop due to the stop and start nature of its jobs. None of the collectors used more than 100MB of memory.

Workload CPU and Memory Utilization. The CPU utilization for the tested workloads when under monitoring and without monitoring (baseline) indicates that the collectors have negligible performance impact on the workloads. Similarly, the collectors did not affect workload memory usage. This is expected as the collectors passively monitor for system calls via kernel probes, unlike system call interposition techniques.

Semantic Compression. Table V shows the number of objects (records) and the file sizes generated by each of the collectors. At first sight, it appears that go-audit is the clear winner in generating the least number of objects among the collectors. However, this exposes a serious underlying problem—go-audit misses

the vast majority of messages, especially in highly intensive benchmarks such as Apache Benchmark. Audit's internal buffer gets overwhelmed and starts spilling messages. This underscores the fact that audit is not the right tool for collecting system call events. Audit functions better as a rule-based runtime monitor whereby one can build rules to monitor specific processes or directories, limiting the scope of observed system calls. Furthermore, audit's lack of container support makes it a less than ideal choice for modern deployments.

When compared to SysFlow, Sysdig generates one order of magnitude more records at the low-end (httpd_prefork) and six orders of magnitude more records at the high-end (databases). SysFlow performs much better on the database benchmarks because these databases are efficient, opening single file handles to each backend files, and leaving them open. Sysdig captures each read and write as individual records, whereas SysFlow creates one file flow per file, reexporting them every 30 seconds with updated statistics.

SysFlow was closest in size to Sysdig on the Apache benchmarks. This is largely due to the fact that these benchmarks create thousands of quick network connections, which induce thousands of flows. Since there are not a lot of reads/writes per connection, the compression is smaller. An interesting aspect of the Apache web server under different multi-processing configurations is that it has a model where a root process or thread accepts a new connection, and then passes that connection handle off to another thread or process for processing. In some configurations, that connection will get passed to a third thread to be closed. In our prototype, we used the thread ID as an attribute to determine uniqueness in both network and file flows. This means that these Apache servers will generate up to three network flows per connection received. This was a conscious decision made for visibility; however, we could make the implementation configurable to only generate one flow when a connection is passed around, thus further reduce our output size.

SysFlow provided over an order of magnitude compression on the Hadoop benchmarks compared to Sysdig and Audit. Hadoop is the most complex benchmark in that it creates tens of thousands of network and file flows as well as file and process events. Indeed, the page rank benchmark used over 700 unique java processes, 34,000 unique files, and 38,000 file events including 2,400 `makedirs`, and 31,000 `unlinks`. Hadoop demonstrates SysFlow's broad ability to compress telemetry data under a diverse set of system calls. The matrix multiplication benchmark was a thread intensive benchmark whereby, one thread was created per row of the resulting matrix. SysFlow achieved high semantic compression by summarizing thread creation/destruction as a process flow.

Object Serialization. Table VI compares the size of SysFlow records serialized in both Avro, JSON (gzip compressed), and Parquet. Notice that even in this small example, compressed JSON is 2–3× the size of Avro. JSON attribute names are encoded in every record as key-value pairs to attribute values, meaning that the length of an attribute name can have a significant impact on file size. Attribute name storage is unnecessary, and is encoded by position in Avro records.

We also exported SysFlow records in Parquet, which is a binary-encoded columnar storage format—it stores records together by column rather than row. This has several advantages for

TABLE VI: SysFlow's serialization format sizes (KB) computed from 1-min benchmark samples, with overhead comparisons to native Avro format (parenthesized)

Benchmark	Avro	JSON (gzip)	Parquet
httpd_prefork	1,270	3,300 (+160%)	1,334 (+5%)
httpd_worker	1,874	4,900 (+161%)	2,041 (+9%)
httpd_event	2,320	5,800 (+150%)	2,567 (+11%)
mysql	75	118 (+57%)	85 (+13%)
postgres	61	87 (+43%)	68 (+11%)
redis	52	93 (+79%)	59 (+13%)
minio	2,939	3,100 (+5%)	3,239 (+10%)
hadoop	11,119	18,483 (+66%)	9,682 (-13%)

* Avro uses deflate codec with 80KB blocks; JSONs are compressed with gzip.

TABLE VII: Summary of a production environment telemetry.

Feature	Findings
Executed binaries (names only)	redis-server, apollo-engine-binary-linux, node, npm, nginx, java, logspout, sh, sed, tr, python3.7, grep, jq, ...
PE operations (#)	clone: 9421, exec: 4585, exit: 7277, setuid: 1041
FE operations (#)	mkdir: 5837, symlink: 4070, unlink: 1740, rename: 906
FF opflags (#)	open: 3517723, read: 3676020, write: 127930, close: 3629040
NF opflags (#)	accept: 169589, connect: 897620, shutdown: 46043, recv: 1360250, send: 1201979, close: 1267324

compression, reading, and data parsing that make it popular for big data applications. Unfortunately, Parquet is not an ideal format for streaming data; however, it may be a viable serialization option once data has been pushed to cloud storage since it is almost as compact as Avro, and is optimized for query-based analytics. We plan to explore the efficacy of this data conversion going forward.

VI. LIVE DEPLOYMENTS

Enterprise Portal. We used the SysFlow stack to monitor a project management portal for a large organization with a thousand daily users and over 100 managed projects. The website is built using a set of docker containers and is composed of a front-end, back-end rest API, cache, and database. All incoming web traffic is received from an external gateway.

The collector was deployed on the production server and SysFlow records were exported to an external S3-compliant object store server in 5-minute intervals. The front-end web server received over 50K web requests, and generated several million read and write operations while serving up pages.

A quick analysis of SysFlow traces provided a list of the different binaries that were executed on these production containers (see Table VII). Of particular interest is the execution of a shell and several command line executables including `grep`, `jq`, `sed` and `tr`. The execution of these command line programs corresponded to a user entering one of the containers from the local host. The traces also reveal the arguments of each command and how they were used in the system.

Cloud DevSecOps. In another setting, we deployed SysFlow on a twelve-node Kubernetes cluster to monitor a web-based enterprise management service hosted on a cloud environment. Our data processing pipeline (§IV-B) was configured with a policy comprising 30 MITRE ATT&CK TTPs, including interactions with sensitive files, process executions, and suspicious network

activity. As part of routine security tests and hardening, an external penetration testing team automated scripts to probe and modify the service's containers, and setup command-and-control (C&C) channels between the containers. We used the collected telemetry to assess both the security posture of the container environment and the development practices that went into building and configuring the containers.

Our analysis uncovered critical security issues with this deployment. For example, our data showed that an ephemeral container was used to export data from one database to another at intervals. Each run, this container re-installed all private keys into key stores, and used clear-text connection strings to authenticate to the databases. Moreover, although the main web server container was executed under an unprivileged user, that user still had write access to modify the application's web files, and compile source files using a `javac` binary left in the unsanitized container image. We were also able to detect the C&C communication patterns between the containers. Our system telemetry stack enabled us to effectively automate the extraction of security-relevant TTPs and incorporate runtime monitoring and attack vector discovery as a step of the development and security operations (DevSecOps) workflow.

VII. CONCLUSION

SysFlow provides deep visibility into computer workloads and enables scalable system analysis. It lifts raw system call information into a higher-level representation that captures process and container behaviors and interactions with other resources on a system. Evaluations on a wide variety of container-based workloads show that SysFlow reduces the telemetry footprint by several orders of magnitude over existing state-of-the-art solutions. Its open format and architecture makes it suitable for the creation of a common platform for system telemetry and analytics that will afford practitioners time to focus on high-value tasks rather than infrastructure building.

REFERENCES

- [1] RedHat, "Linux audit framework," <https://people.redhat.com/sgrubb/audit/>, 2006, acc. 2019-09-16.
- [2] Sysdig, "Sysdig," <https://www.sysdig.com>, 2019, acc. 2019-09-16.
- [3] S. Ma, J. Zhai, Y. Kwon, K. H. Lee, X. Zhang, G. Ciocarlie, A. Gehani, V. Yegneswaran, D. Xu, and S. Jha, "Kernel-supported cost-effective audit logging for causality tracking," in *USENIX ATC*, 2018.
- [4] S. Ma, X. Zhang, and D. Xu, "ProTracer: Towards practical provenance tracing by alternating between logging and tainting," in *NDSS*, 2016.
- [5] Cisco, "Introduction to Cisco IOS NetFlow—A Technical Overview," https://www.cisco.com/c/en/us/products/collateral/ios-nx-os-software/ios-netflow/prod_white_paper0900aecd80406232.html, 2012, acc. 2020-11-09.
- [6] MITRE, "MITRE ATT&CK Matrix for Enterprise Linux," <https://attack.mitre.org/matrices/enterprise/linux/>, 2019, acc. 2020-05-24.
- [7] G. Maier, R. Sommer, H. Dreger, A. Feldmann, V. Paxson, and F. Schneider, "Enriching network security analysis with time travel," in *ACM SIGCOMM Conference on Data Communication*, 2008.
- [8] V. Paxson, "Bro: A system for detecting network intruders in real-time," *The International Journal of Computer and Telecommunications Networking*, vol. 31, no. 23-24, Dec. 1999.
- [9] Y. Li, R. Miao, C. Kim, and M. Yu, "Flowradar: A better netflow for data centers," in *USENIX NSDI*, 2016.
- [10] C. Estan, K. Keys, D. Moore, and G. Varghese, "Building a Better NetFlow," in *ACM SIGCOMM*, 2004.
- [11] T. Taylor, S. Coull, F. Monroe, and J. McHugh, "Toward efficient querying of compressed network payloads," in *USENIX ATC*, 2012.
- [12] L. Deri, "nprobe: an open source netflow probe for gigabit networks," in *Terena TNC*, 01 2003.
- [13] A. Gehani and D. Tariq, "Spade: Support for provenance auditing in distributed environments," in *ACM/IFIP Middleware*, 2012.
- [14] Y. Ji, S. Lee, E. Downing, W. Wang, M. Fazzini, T. Kim, A. Orso, and W. Lee, "RAIN: Refinable attack investigation with on-demand inter-process information flow tracking," in *ACM CCS*, 2017.
- [15] F. Oliveira, S. Suneja, S. Nadgowda, P. Nagpurkar, and C. Isci, "Opvis: Extensible, cross-platform operational visibility and analytics for cloud," in *ACM/IFIP Middleware: Industrial Track*, 2017.
- [16] K. H. Lee, X. Zhang, and D. Xu, "Loggc: garbage collecting audit log," in *ACM CCS*, 2013.
- [17] R. Madhumathi, "The relevance of container monitoring towards container intelligence," in *ICCCNT*, 2018.
- [18] T. Marian, H. Weatherspoon, K.-S. Lee, and A. Sagar, "Fmeter: Extracting Indexable Low-Level System Signatures by Counting Kernel Function Calls," in *ACM/IFIP Middleware*, 2012.
- [19] Y. Chen, S. Zhang, Q. Guo, L. Li, R. Wu, and T. Chen, "Deterministic replay: A survey," *ACM Computer Surveys*, 2015.
- [20] D. Geels, G. Altekar, S. Shenker, and I. Stoica, "Replay debugging for distributed applications," in *USENIX ATC*, 2006.
- [21] osquery, "osquery," <https://osquery.io/>, 2006, acc. 2019-09-16.
- [22] S. Ma, J. Zhai, F. Wang, K. H. Lee, X. Zhang, and D. Xu, "MPI: Multiple perspective attack investigation with semantics aware execution partitioning," in *USENIX Security*, 2017.
- [23] K. H. Lee, X. Zhang, and D. Xu, "High accuracy attack provenance via binary-based execution partitioning," in *NDSS*, 2013.
- [24] Z. Xu, Z. Wu, Z. Li, K. Jee, J. Rhee, X. Xiao, F. Xu, H. Wang, and G. Jiang, "High fidelity data reduction for big data security dependency analyses," in *ACM CCS*, 2016.
- [25] Y. Tang, D. Li, Z. Li, M. Zhang, K. Jee, X. Xiao, Z. Wu, J. Rhee, F. Xu, and Q. Li, "Nodemerge: Template based efficient data reduction for big-data causality analysis," in *ACM CCS*, 2018.
- [26] DARPA, "Transparent Computing," <https://www.darpa.mil/program/transparent-computing>, 2014, acc. 2022-05-26.
- [27] G. Jenkinson, L. Carata, N. Balakrishnan, T. Bytheway, R. Sohan, R. N. M. Watson, J. Anderson, B. Kidney, A. Strnad, A. Thomas, and G. Neville-Neil, "Applying provenance in apt monitoring and analysis: Practical challenges for scalable, efficient and trustworthy distributed provenance," in *USENIX TAPP*, 2017.
- [28] X. Shu, F. Araujo, D. L. Schales, M. P. Stoecklin, J. Jang, H. Huang, and J. R. Rao, "Threat intelligence computing," in *ACM CCS*, 2018.
- [29] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. Venkatakrishnan, "Holmes: Real-time apt detection through correlation of suspicious information flows," in *IEEE S&P*, 2018.
- [30] M. N. Hossain, S. M. Milajerdi, J. Wang, B. Eshete, R. Gjomemo, R. Sekar, S. Stoller, and V. Venkatakrishnan, "SLEUTH: Real-time attack scenario reconstruction from COTS audit data," in *USENIX Security*, 2017.
- [31] MITRE, "Cve-2017-5941," <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-5941>, 2017, acc. 2019-09-16.
- [32] Kubernetes, "Production-grade container orchestration - Kubernetes," <https://kubernetes.io>, 2019, acc. 2019-09-17.
- [33] Docker, "Swarm: a Docker-native clustering system," <https://github.com/docker/swarm>, 2019, acc. 2019-09-17.
- [34] OpenShift, "OpenShift: Container Application Platform by Red Hat, Built on Docker and Kubernetes," <https://www.openshift.com/>, 2019.
- [35] The Apache Software Foundation, "Apache avro," <https://avro.apache.org/>, 2012, acc. 2019-09-16.
- [36] Kubernetes Community, "Kubernetes: Production-grade container orchestration," <https://kubernetes.io/>, 2019, acc. 2019-09-16.
- [37] Cloud Native Computing Foundation, "Helm: The package manager for kubernetes," <https://helm.sh>, 2019, acc. 2019-09-16.
- [38] Slack Technologies Inc., "go-audit," <https://github.com/slackhq/go-audit>, 2016, acc. 2019-09-17.
- [39] Red Hat, "Ansible," <https://www.ansible.com/>, 2019, acc. 2020-11-09.
- [40] T. P. Robitaille, <https://github.com/astrofrog/psrecord>, 2013, acc. 2020-11-09.
- [41] Apache HTTP Server Project, "ab - apache http server benchmarking tool," <https://httpd.apache.org/docs/2.4/programs/ab.html>, 2019, acc. 2019-09-17.
- [42] Wasabi, "Wasabi s3 benchmark," <https://github.com/wasabi-tech/s3-benchmark>, 2019, acc. 2019-09-17.
- [43] HammerDB, "Hammerdb tpc benchmarks," <https://www.hammerdb.com/>, 2019, acc. 2019-09-17.
- [44] G. Chenzhao, "Hibench big data benchmark suite," <https://github.com/Intel-bigdata/HiBench>, 2019, acc. 2019-09-17.
- [45] C. Maigre, "Multithreading benchmarks," <https://github.com/CorkyMaigre/multithreading-benchmarks>, 2016, acc. 2020-05-20.